Programación Orientada a Objetos
2026 - 2do cuatrimestre - Miércoles tarde

Trabajo Práctico Obligatorio

Etapa 1: Modelado y diseño del sistema

Propuesta B: Clínica veterinaria

Integrantes:

Bustamante, Enrique

Franco, Tomás

Suárez, Elian

Repositorio: https://github.com/enriquebustamantebmw-hash/TPOVeterinariaPOO

1. Propuesta seleccionada e integrantes

Elegimos la Propuesta B, la clínica veterinaria. La clínica tiene varios profesionales y quiere informatizar la atención de los pacientes, los turnos y el control de insumos. El sistema tiene que permitir registrar a los dueños y sus mascotas (de distintas especies), manejar la agenda de cada veterinario sin que se superpongan los turnos, llevar la historia clínica de cada mascota, controlar el stock de medicamentos e insumos, facturar y mandar recordatorios de vacunas.

Integrantes del grupo:

- Bustamante, Enrique
- Franco, Tomás
- Suárez, Elian

2. Análisis del enunciado

2.1 Actores

- **Recepcionista:** registra dueños y mascotas, da, reprograma y cancela turnos, carga urgencias y ausencias, y factura. No puede cargar diagnósticos ni tratamientos.
- **Veterinario:** atiende las consultas y carga diagnósticos, tratamientos y vacunas. También consulta la historia clínica. Cada uno tiene su horario y alguno puede estar de guardia.
- **Dueño de la mascota:** puede pedir y cancelar turnos y recibe las notificaciones y los recordatorios.
- **Administrador:** maneja el stock, da de alta a los profesionales con sus horarios y recibe las alertas de stock bajo.
- **Tiempo:** lo agregamos como actor secundario porque los recordatorios de vacunas los dispara el sistema solo, una vez por día.

2.2 Requerimientos funcionales

N°	Requerimiento
RF01	Registrar, modificar y consultar dueños.
RF02	Registrar mascotas de distintas especies (perro, gato, ave, conejo) con los datos de cada una.
RF03	Transferir una mascota a otro dueño sin perder la historia clínica.
RF04	Asignar turnos (mascota, profesional, fecha y hora, tipo de atención) controlando horario y superposición.
RF05	Reprogramar y cancelar turnos aplicando la política de cancelación.
RF06	Registrar urgencias en el primer horario del profesional de guardia, desplazando turnos si hace falta.
RF07	Registrar ausencias y pedir seña cuando un dueño tiene tres ausencias seguidas.
RF08	Registrar consultas con diagnósticos, tratamientos y vacunas en la historia clínica.
RF09	Consultar la historia clínica de una mascota.
RF10	Controlar el stock de medicamentos e insumos (alta, reposición y consumo).
RF11	Avisar al administrador cuando un insumo queda por debajo del mínimo.
RF12	Facturar con distintos medios de pago y descuentos.
RF13	Mandar recordatorios de vacunas próximas a vencer.
RF14	Administrar los profesionales, sus horarios y la guardia.

2.3 Reglas de negocio

N°	Regla
RN01	La duración del turno depende del tipo: consulta general 30 min, vacunación 15 min y cirugía 120 min.
RN02	Cada profesional atiende dentro de su horario (por ejemplo, lunes a sábado de 9 a 19).
RN03	Un profesional no puede tener dos turnos superpuestos.
RN04	Se puede cancelar sin cargo hasta 2 horas antes. Después se cobra el 20% de la consulta.
RN05	Si alguien falta sin avisar, el turno queda como ausente.
RN06	Con tres ausencias seguidas hay que pagar una seña para pedir turnos nuevos.
RN07	Una urgencia toma el primer horario del profesional de guardia. Si está ocupado, el turno siguiente se reprograma y se le avisa al dueño.
RN08	Toda mascota tiene un dueño responsable y un dueño puede tener varias mascotas.
RN09	Si una mascota pasa a otro dueño, conserva su historia clínica.
RN10	Cada especie tiene sus datos propios y su propio calendario de vacunación.
RN11	Solo un veterinario puede registrar diagnósticos y tratamientos. Recepción no puede.
RN12	Cuando el stock de un insumo baja del mínimo se genera una alerta.
RN13	Se puede pagar con distintos medios de pago y aplicar descuentos.

2.4 Supuestos

Hay cosas que el enunciado no aclara, así que tomamos estas decisiones:

1. La urgencia dura 30 minutos, igual que una consulta general, pero tiene un recargo en el precio.
2. Hay un solo veterinario de guardia por franja horaria y lo define el administrador.
3. Las tres ausencias seguidas se cuentan por dueño y no por mascota. Si el dueño asiste a un turno o paga la seña, el contador vuelve a cero.
4. El 20% de la cancelación tardía se calcula sobre el precio del tipo de atención del turno y se cobra con una factura aparte.
5. La seña es un porcentaje del precio del turno (configurable) y después se descuenta de la factura de esa atención.
6. Cuando una urgencia desplaza un turno, ese turno pasa al primer horario libre del mismo veterinario. Si al dueño no le sirve, lo puede reprogramar o cancelar sin cargo.
7. Los medios de pago son efectivo, tarjeta (débito o crédito en cuotas) y transferencia. Los descuentos pueden ser porcentuales (por ejemplo, jubilados o convenios) o de monto fijo, y se aplica uno solo por factura.
8. Los recordatorios salen una vez por día para las vacunas que vencen en los próximos 7 días, por mail o WhatsApp.

9. Los insumos que se usan en una consulta se descuentan del stock al cargar la consulta. El stock mínimo de cada insumo lo carga el administrador.
10. El dueño puede pedir y cancelar turnos por su cuenta (por ejemplo, desde una web). Todo lo demás lo hace el personal de la clínica.

También tuvimos en cuenta algunos requisitos no funcionales: que cada usuario entre con su rol (para poder controlar la RN11), que se puedan agregar especies, tipos de atención, medios de pago o canales de aviso sin tocar lo que ya existe, y que los datos queden guardados en forma persistente.

3. Casos de uso

3.1 Diagrama de casos de uso

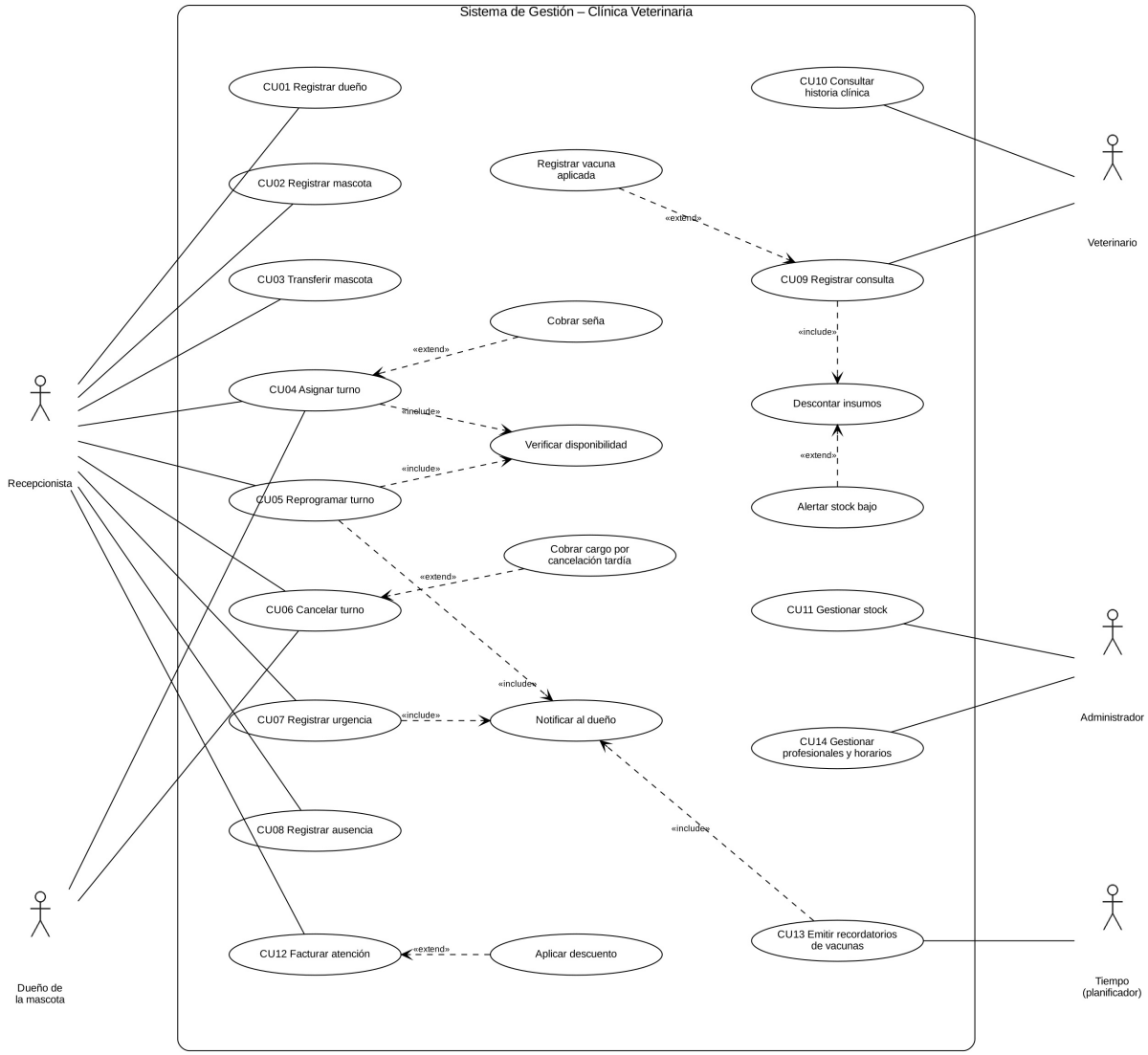


Figura 1: Diagrama de casos de uso

3.2 Descripción de los casos de uso

CU01 - Registrar dueño

Actor	Recepcionista
Precondición	El recepcionista inició sesión.
Flujo principal	1. El recepcionista carga DNI, nombre, apellido, teléfono, mail y dirección. 2. El sistema controla que el DNI no esté cargado. 3. El sistema guarda el dueño.
Flujos alternativos	2a. El DNI ya existe: se muestra el dueño cargado y se puede editar. 2b. Falta algún dato obligatorio: se avisa y no se guarda.

CU02 - Registrar mascota

Actor	Recepcionista
Precondición	El dueño ya está registrado.
Flujo principal	1. Se selecciona el dueño. 2. Se elige la especie (perro, gato, ave o conejo).

	3. Se cargan los datos comunes y los propios de la especie. 4. El sistema crea la mascota con su historia clínica vacía y el calendario de vacunación de su especie.
Flujos alternativos	3a. Faltan datos de la especie: se vuelven a pedir.

CU03 - Transferir mascota

Actor	Recepcionista
Precondición	La mascota y el nuevo dueño están registrados.
Flujo principal	1. Se elige la mascota y el nuevo dueño. 2. El sistema saca la mascota del dueño anterior y la asocia al nuevo. 3. La historia clínica queda igual.
Flujos alternativos	1a. El nuevo dueño no existe: se hace el CU01 y se sigue. 2a. La mascota tiene turnos a futuro: se mantienen y los avisos pasan al nuevo dueño.

CU04 - Asignar turno

Actor	Recepcionista, Dueño
Precondición	La mascota y el profesional están registrados.
Flujo principal	1. Se indica mascota, profesional, fecha y hora, y tipo de atención. 2. El sistema controla que el dueño no tenga que pagar seña. 3. Se verifica la disponibilidad (horario del profesional, duración según el tipo y que no se superponga). 4. Se crea el turno en estado PROGRAMADO. 5. Se le avisa al dueño.
Flujos alternativos	2a. El dueño tiene tres ausencias seguidas y no pagó la seña: se cobra la seña (extend). Si no paga, no se da el turno. 3a. El horario está fuera de la franja o ya está ocupado: se avisa y se propone el primer horario libre.

CU05 - Reprogramar turno

Actor	Recepcionista
Precondición	Hay un turno PROGRAMADO.
Flujo principal	1. Se elige el turno y el nuevo horario. 2. Se verifica la disponibilidad. 3. Se actualiza el turno. 4. Se le avisa al dueño.
Flujos alternativos	2a. El horario no está disponible: se propone otro.

CU06 - Cancelar turno

Actor	Recepcionista, Dueño
Precondición	Hay un turno PROGRAMADO.
Flujo principal	1. Se elige el turno. 2. El sistema controla que falten más de 2 horas. 3. El turno pasa a CANCELADO sin cargo y el horario queda libre.
Flujos alternativos	2a. Faltan 2 horas o menos: se cobra el 20% de la consulta (extend) y después se cancela.

CU07 - Registrar urgencia

Actor	Recepcionista
Precondición	Hay un veterinario de guardia.
Flujo principal	1. Se indica la mascota. 2. El sistema busca al veterinario de guardia y su primer horario. 3. Si está libre, crea el turno de urgencia.
Flujos alternativos	3a. El horario está ocupado: se crea la urgencia igual, los turnos que se pisan se pasan al siguiente horario libre y se avisa a cada dueño.

CU08 - Registrar ausencia

Actor	Recepcionista
Precondición	Hay un turno PROGRAMADO cuyo horario ya pasó y el dueño no vino.

Flujo principal	1. Se marca el turno como ausente. 2. Se suma una ausencia seguida al dueño.
Flujos alternativos	2a. Llega a tres ausencias seguidas: el dueño queda marcado para pagar seña.

CU09 - Registrar consulta

Actor	Veterinario
Precondición	El usuario inició sesión y la mascota está registrada.
Flujo principal	1. Se elige la mascota. 2. El sistema controla que el usuario sea veterinario. 3. Se cargan motivo, diagnósticos y tratamientos. 4. La consulta se agrega a la historia clínica. 5. Se descuentan los insumos usados (include). 6. El turno pasa a ATENDIDO.
Flujos alternativos	2a. El usuario no es veterinario: se rechaza (RN11). 3a. Se aplicó una vacuna: se registra y se calcula el vencimiento según la especie (extend). 5a. Algún insumo queda bajo el mínimo: se genera la alerta (extend). 5b. No hay stock suficiente: se avisa y la consulta se guarda igual.

CU10 - Consultar historia clínica

Actor	Veterinario
Precondición	La mascota está registrada.
Flujo principal	1. Se busca la mascota. 2. El sistema muestra consultas, diagnósticos, tratamientos y vacunas ordenados por fecha, incluso los de dueños anteriores.
Flujos alternativos	1a. La mascota no existe: se avisa.

CU11 - Gestionar stock

Actor	Administrador
Precondición	El administrador inició sesión.
Flujo principal	1. Se da de alta, se modifica o se repone un insumo o medicamento, con su stock mínimo. 2. Se actualiza el stock.
Flujos alternativos	1a. El código ya existe: se avisa. 2a. El medicamento está vencido: se da de baja.

CU12 - Facturar atención

Actor	Recepcionista
Precondición	Hay un turno ATENDIDO sin facturar o un cargo pendiente (seña o cancelación tardía).
Flujo principal	1. Se elige el turno. 2. El sistema arma los ítems (atención e insumos). 3. Se elige el medio de pago. 4. Se calcula el total, se cobra y se emite la factura.
Flujos alternativos	3a. Corresponde un descuento: se aplica (extend). 4a. El pago es rechazado: se pide otro medio de pago. 4b. El dueño ya había pagado seña: se descuenta del total.

CU13 - Emitir recordatorios de vacunas

Actor	Tiempo
Precondición	Hay vacunas aplicadas cargadas.
Flujo principal	1. Todos los días el sistema busca las vacunas que vencen en los próximos 7 días. 2. Se le avisa a cada dueño (include Notificar al dueño).
Flujos alternativos	1a. No hay vacunas por vencer: no se manda nada. 2a. Falla el envío: queda registrado y se reintenta al otro día.

CU14 - Gestionar profesionales y horarios

Actor	Administrador

Precondición	El administrador inició sesión.
Flujo principal	1. Se da de alta o se modifica un veterinario, sus días y horarios, y si está de guardia. 2. Se guardan los cambios.
Flujos alternativos	1a. El nuevo horario deja turnos afuera: el sistema los lista para reprogramarlos (CU05).

4. Diagrama de clases inicial

Esta es la primera versión, armada directamente a partir del enunciado. Tiene dos jerarquías de herencia con sobreescritura de métodos. Por un lado, Persona, de la que heredan Dueno y Empleado, y de Empleado heredan Veterinario, Recepcionista y Administrador, que sobrescriben getRol(). Por otro lado, Mascota, con las subclases Perro, Gato, Ave y Conejo, que sobrescriben getVacunasObligatorias() y getDatosEspecificos().

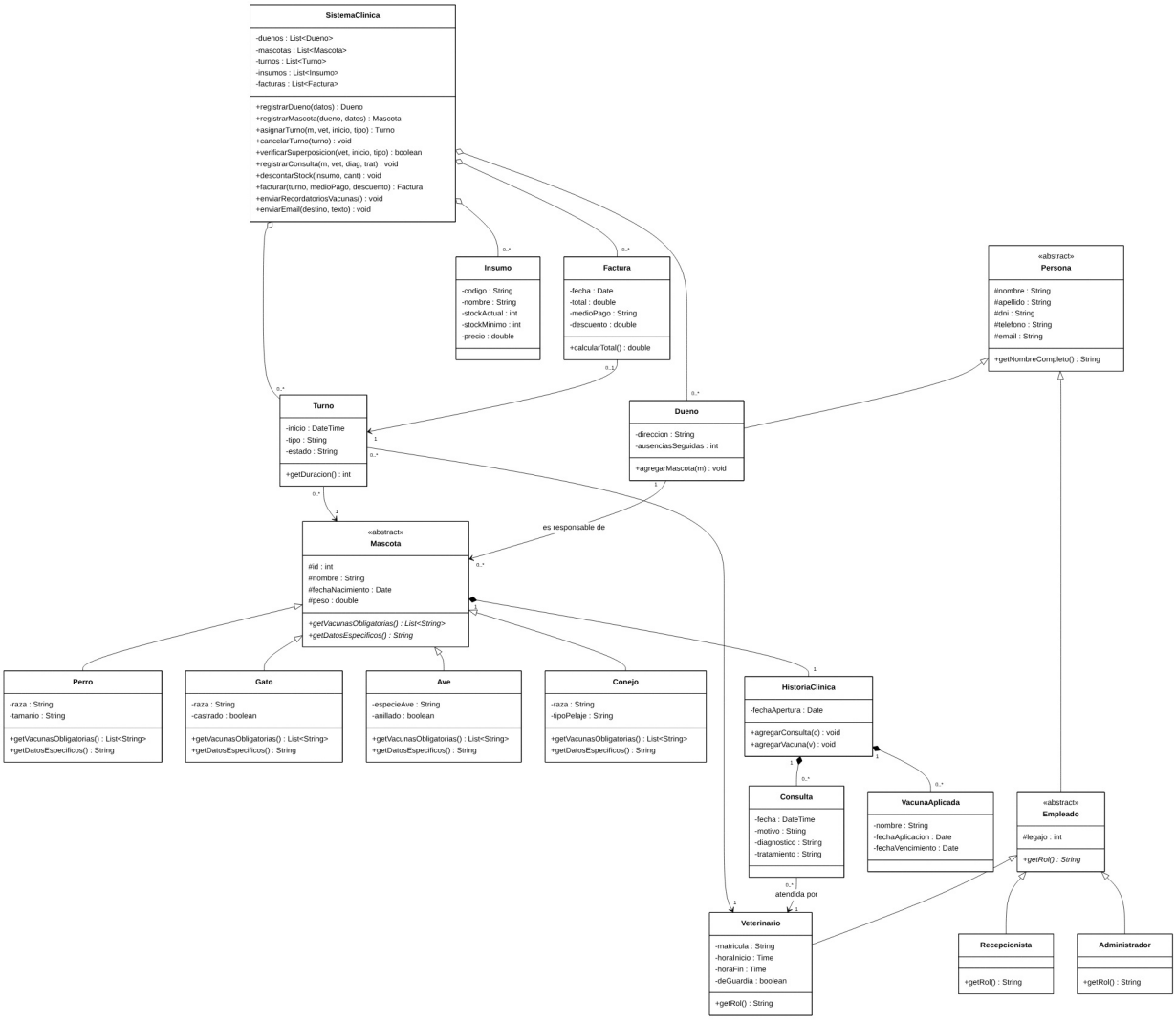


Figura 2: Diagrama de clases inicial

Cuando revisamos esta versión encontramos varios problemas, que son los que después tratamos de resolver con los principios y patrones:

- SistemaClinica hace prácticamente todo (turnos, consultas, stock, facturación y mails). Es una clase demasiado grande y con poca cohesión.
- El tipo de turno, el medio de pago y el estado son String, así que la duración y los cálculos se tienen que resolver con if o switch. Para agregar una opción nueva hay que modificar código que ya funciona.
- La verificación de superposición está en SistemaClinica, lejos de los datos que necesita (los turnos y el horario del veterinario).
- El envío de mails depende de una implementación concreta.
- No hay forma de controlar que solo un veterinario cargue diagnósticos.
- El diagnóstico y el tratamiento son solo textos dentro de la consulta.

5. Diagramas de secuencia

5.1 CU04 - Asignar turno

Elegimos este caso porque tiene flujos alternativos de error. Hay un primer alt para el caso en que el dueño tenga que pagar seña y otro para cuando el horario no está disponible. Además hay un loop que recorre los turnos del día para ver si alguno se superpone.

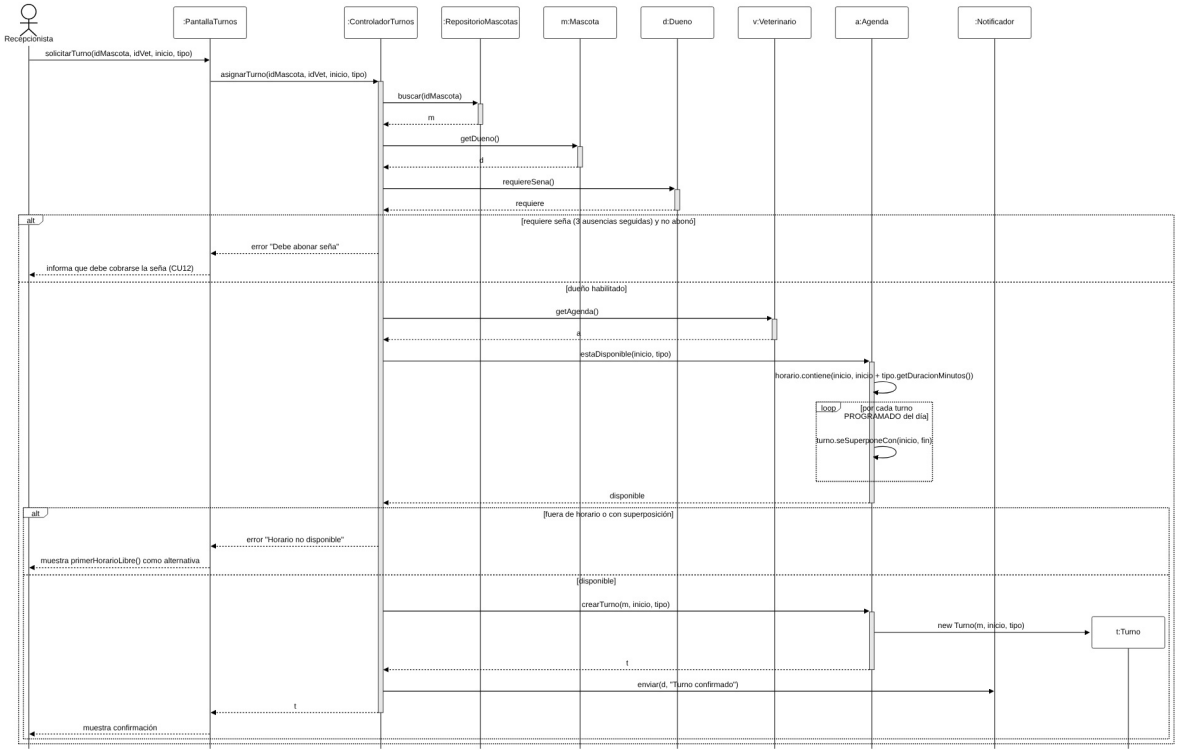


Figura 3: Diagrama de secuencia de Asignar turno

5.2 CU09 - Registrar consulta

En este diagrama se ve el control de que el usuario sea veterinario (RN11), el registro de la consulta en la historia clínica, la vacuna como opcional (opt), el descuento de cada insumo usado (loop) y la alerta cuando el stock queda bajo el mínimo (opt).

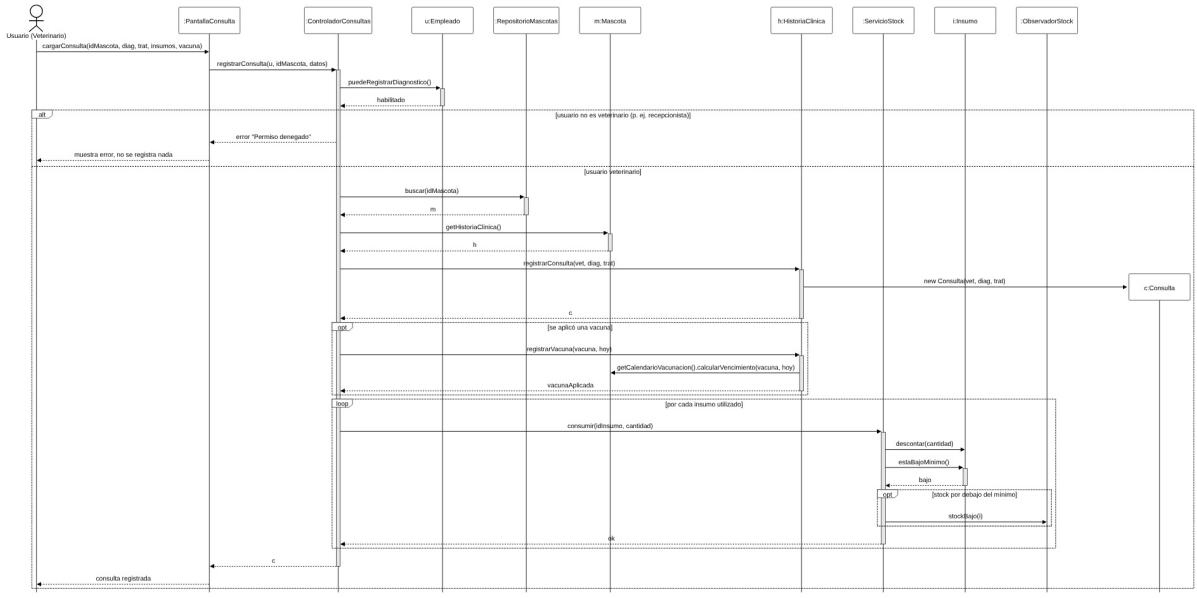


Figura 4: Diagrama de secuencia de Registrar consulta

6. Aplicación de principios SOLID y patrones GRASP

6.1 Principios SOLID

Responsabilidad única (SRP)

En el diseño inicial, SistemaClinica registraba dueños, daba turnos, controlaba superposiciones, cargaba consultas, descontaba stock, facturaba y mandaba mails. Cualquier cambio, por ejemplo en la política de cancelación, obligaba a tocar esa clase, con el riesgo de romper otras partes que no tenían nada que ver.

Decidimos dividirla según cada área. Quedaron ControladorTurnos, ControladorConsultas y ControladorFacturacion, dos servicios (ServicioStock y ServicioRecordatorios) y repositorios para el acceso a los datos (RepositorioTurnos, RepositorioMascotas y RepositorioInsumos).

La ventaja es que ahora cada clase tiene un solo motivo para cambiar. Son más chicas, se entienden mejor y se pueden probar por separado.

Abierto/Cerrado (OCP)

El tipo de atención, el medio de pago y el descuento estaban como String o double y se resolvían con switch. Si mañana se agrega un "control post quirúrgico" o pagar con Mercado Pago, había que modificar Turno y Factura.

Lo resolvimos modelando cada cosa que puede variar como una abstracción con sus subclases: la clase abstracta TipoAtencion (ConsultaGeneral, Vacunacion, Cirugia, Urgencia) y las interfaces MedioPago (Efectivo, Tarjeta, Transferencia) y Descuento (SinDescuento, DescuentoPorcentual, DescuentoMontoFijo). Lo mismo pasa con Mascota y sus especies.

Así, para agregar una opción nueva solo hay que crear una clase, sin tocar Turno, Agenda ni Factura, y es menos probable meter errores en lo que ya funciona.

Inversión de dependencias (DIP)

SistemaClinica.enviarEmail() dependía de una forma concreta de mandar mails, y la lógica usaba directamente las listas en memoria. Pasar a WhatsApp o a una base de datos implicaba reescribir la lógica.

Hicimos que las clases de alto nivel dependan de interfaces y que las implementaciones se les pasen desde afuera. Creamos la interfaz Notificador (con NotificadorEmail y NotificadorWhatsApp), las interfaces de repositorio y ObservadorStock (implementada por AlertaAdministrador). Los controladores y servicios solo conocen esas interfaces.

De esta forma se puede cambiar el canal de aviso o la forma de guardar los datos sin modificar la lógica de negocio, y para las pruebas se pueden usar implementaciones falsas.

6.2 Patrones GRASP

Experto en información

En la primera versión, SistemaClinica controlaba superposiciones, calculaba el total de la factura y buscaba las vacunas por vencer, aunque los datos para hacerlo estaban en otras clases. Eso generaba mucho acoplamiento.

Pasamos cada responsabilidad a la clase que tiene la información necesaria. Turno sabe si se superpone con otro horario (seSuperponeCon) y cuánto hay que cobrar si se cancela tarde (cancelar). Agenda sabe si un horario está disponible porque tiene los turnos del veterinario, y HorarioAtencion sabe si un horario entra en la franja. Dueno sabe si tiene que pagar seña porque lleva la cuenta de sus ausencias. Factura calcula su total, Insumo sabe si está bajo el mínimo y HistoriaClinica sabe qué vacunas están por vencer.

Así la lógica queda junto a los datos que usa, se respeta el encapsulamiento y cada regla de negocio queda en un solo lugar.

Creador

En el diseño inicial no estaba claro quién creaba las consultas, las vacunas, los turnos y los ítems de factura. Todo salía de SistemaClinica.

Aplicamos Creador: la clase que contiene a otra, o que tiene los datos para inicializarla, es la que la crea. HistoriaClinica crea Consulta y VacunaAplicada porque las contiene. Agenda crea los Turno porque los guarda y conoce el horario. Factura crea sus ItemFactura y Consulta crea Diagnostico y Tratamiento.

Esto mantiene bajo el acoplamiento, porque el creador ya estaba relacionado con lo que crea, y asegura que cada objeto se cree asociado a su contenedor y en un estado válido.

Controlador

Sin un punto de entrada claro, la interfaz de usuario terminaría manejando directamente las clases del dominio y mezclando la pantalla con las reglas de negocio.

Agregamos controladores que reciben los eventos del sistema y coordinan a los objetos del dominio, pero sin tener ellos la lógica. ControladorTurnos se encarga de los casos de uso CU04 a CU08, ControladorConsultas de CU09 y CU10, y ControladorFacturacion de CU12. ControladorConsultas además controla el rol con Empleado.puedeRegistrarDiagnostico() antes de dejar cargar una consulta.

La ventaja es que se separa la presentación del dominio y la misma lógica se puede usar desde distintas pantallas (recepción o la web del dueño).

Polimorfismo

Varias cosas cambiaban según el tipo: la duración según el tipo de atención, el calendario de vacunación según la especie, cómo se procesa el pago según el medio y los permisos según el rol. Todo eso se resolvía con condicionales.

Usamos métodos polimórficos definidos en una clase abstracta o en una interfaz, y cada subtipo los sobrescribe. TipoAtencion.getDuracionMinutos() devuelve 30, 15, 120 o 30 según la subclase. Mascota.getCalendarioVacunacion() y getDatosEspecificos() se sobrescriben en Perro, Gato, Ave y Conejo. MedioPago tiene procesarPago() y recargo(), Descuento tiene aplicar() y Empleado tiene puedeRegistrarDiagnostico(), que Veterinario sobrescribe para devolver true.

Así se sacan los if repetidos, el código se lee mejor y es lo que hace posible cumplir con Abierto/Cerrado: agregar un subtipo no afecta a las clases que lo usan.

7. Diagrama de clases final

Como el diagrama completo quedaba muy grande, lo separamos en dos partes del mismo modelo. La primera tiene el dominio (personas, mascotas, historia clínica y turnos) y la segunda los controladores, servicios, stock y facturación.

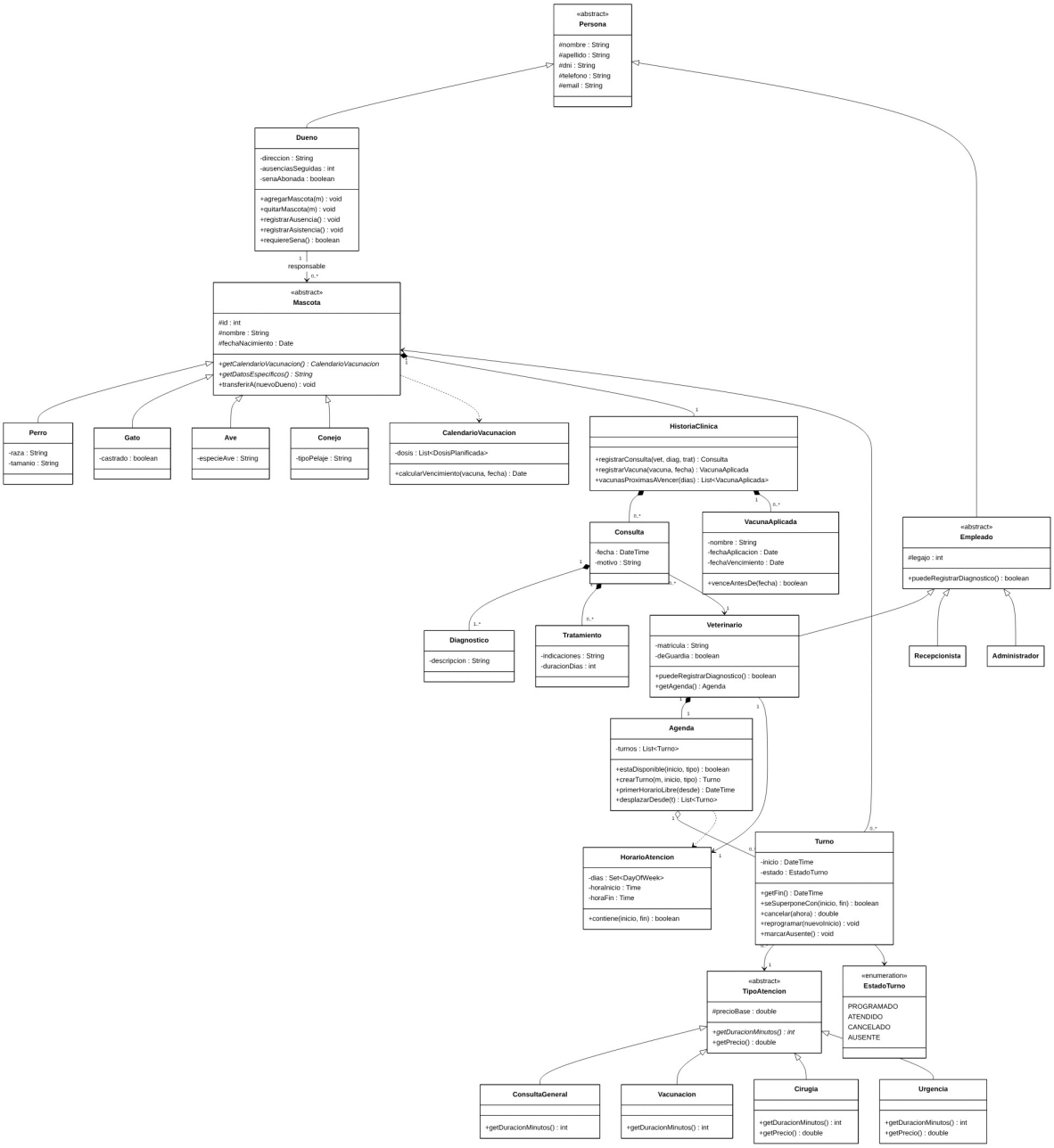


Figura 5: Diagrama de clases final, parte 1 (dominio)

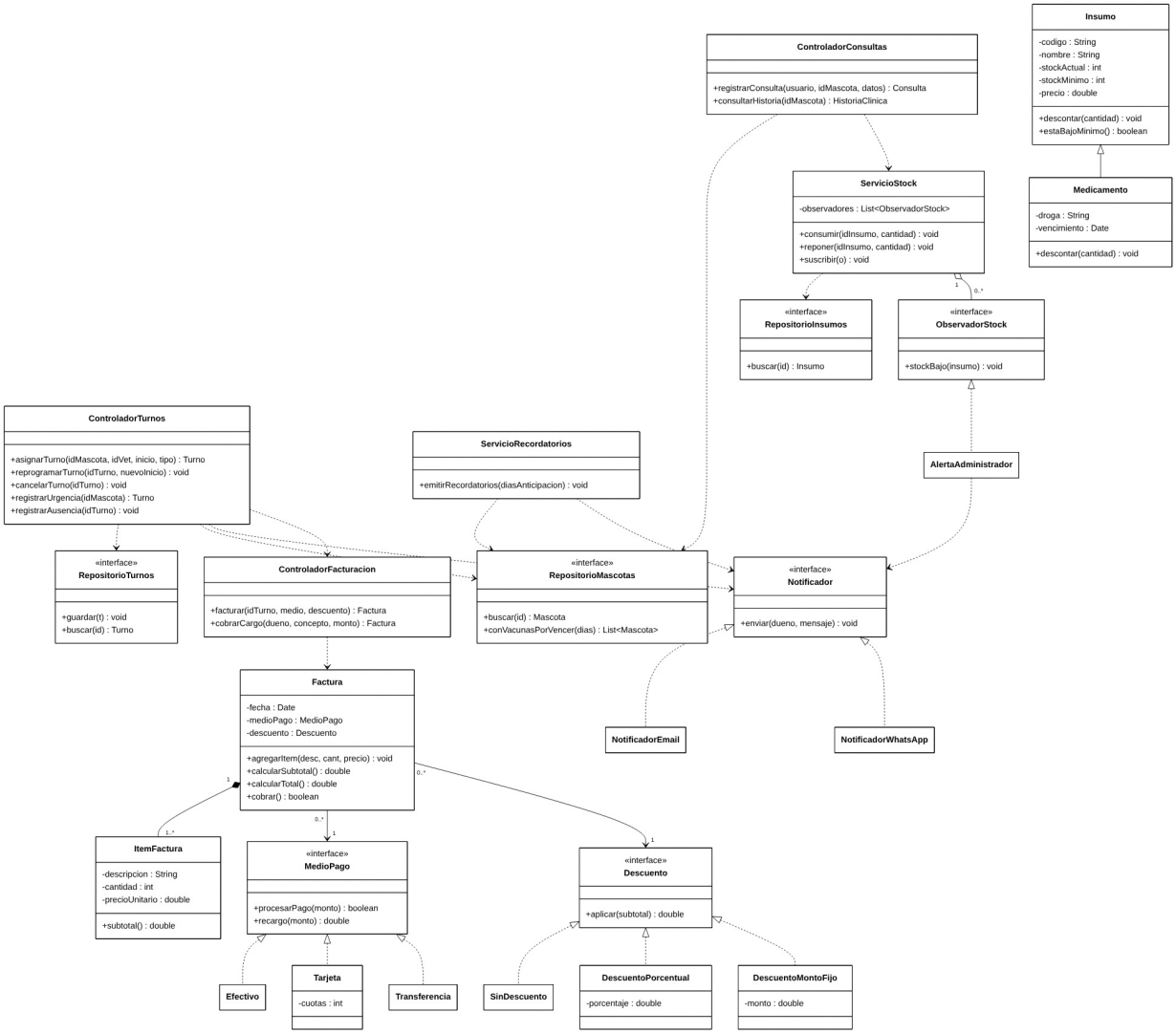


Figura 6: Diagrama de clases final, parte 2 (controladores, servicios, stock y facturación)

7.1 Cambios respecto del diagrama inicial

N°	Qué cambió	Por qué
1	Sacamos SistemaClinica y la reemplazamos por ControladorTurnos, ControladorConsultas, ControladorFacturacion, ServicioStock y ServicioRecordatorios.	SRP y Controlador. Cada clase tiene una sola responsabilidad.
2	Agregamos las interfaces RepositorioTurnos, RepositorioMascotas y RepositorioInsumos.	DIP. La lógica no depende de cómo se guardan los datos.
3	El tipo del turno dejó de ser un String y pasó a ser la jerarquía TipoAtencion (ConsultaGeneral, Vacunacion, Cirugia, Urgencia) con getDuracionMinutos().	OCP y Polimorfismo. Cumple la RN01 sin switch.
4	El estado del turno pasó a ser el enum EstadoTurno.	Evita valores inválidos.
5	Agregamos Agenda (una por veterinario) y HorarioAtencion. La verificación de superposición pasó a Agenda.estaDisponible() y Turno.seSuperponeCon().	Experto y Creador. Cumple RN02 y RN03.
6	Turno ahora tiene cancelar(), reprogramar() y marcarAusente(). Dueno tiene requiereSena(), registrarAusencia() y registrarAsistencia().	Experto. Cumple RN04, RN05 y RN06.
7	getVacunasObligatorias() se cambió por getCalendarioVacunacion(), que devuelve un CalendarioVacunacion que calcula los vencimientos. Agregamos transferirA() en Mascota.	Polimorfismo y Experto. Cumple RN09 y RN10.
8	Diagnostico y Tratamiento pasaron a ser clases que forman parte de Consulta. HistoriaClinica crea las consultas y las vacunas.	Creador. Modela mejor el dominio.
9	En Empleado cambiamos getRol() por puedeRegistrarDiagnostico(), que Veterinario sobrescribe. El controlador lo consulta antes de registrar.	Polimorfismo. Cumple RN11.
10	Factura usa las interfaces MedioPago y Descuento y tiene ItemFactura.	OCP, Polimorfismo, Creador y

		Experto. Cumple RN13.
11	Los avisos pasan por la interfaz Notificador (NotificadorEmail y NotificadorWhatsApp).	DIP. Se puede cambiar el canal sin tocar la lógica.
12	Agregamos ServicioStock con observadores (ObservadorStock y AlertaAdministrador) y Medicamento como subclase de Insumo, que sobrescribe descontar() para controlar el vencimiento.	SRP, DIP y Experto. Cumple RN12.